

视听信息系统导论

优化算法



- 无约束优化
- 高级梯度优化算法

- 无约束最小化

$$\text{minimize } f(x)$$

- f 是凸函数, 二阶连续可微
- 假设在 x^* 取得一个最优值解 (且有限) $p^* = \inf_x f(x)$

- 无约束最小化方法

- 产生点的序列 $x^{(k)} \in \text{dom } f$, $k = 0, 1, \dots$ with
$$f(x^{(k)}) \rightarrow p^*$$

- 可以解释为求解最优条件的迭代方法

$$\nabla f(x^*) = 0$$

- 下降方法

- 选择初始点 $x^{(0)} \in \text{dom } f$, 重复:

$$x^{(k+1)} = x^{(k)} + t^{(k)} \Delta x^{(k)}, \quad k = 1, 2, 3, \dots$$

在优化到一定程度后停下来

- Δx 是优化方向; t 是步长

General descent method.

given a starting point $x \in \text{dom } f$.

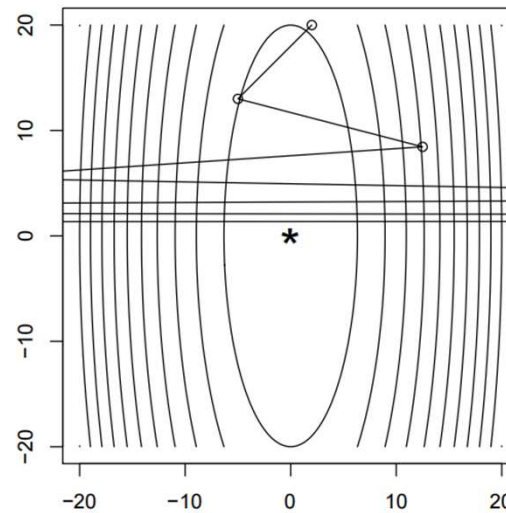
repeat

1. Determine a descent direction Δx .
2. *Line search.* Choose a step size $t > 0$.
3. *Update.* $x := x + t\Delta x$.

until stopping criterion is satisfied.

• 如何选择步长?

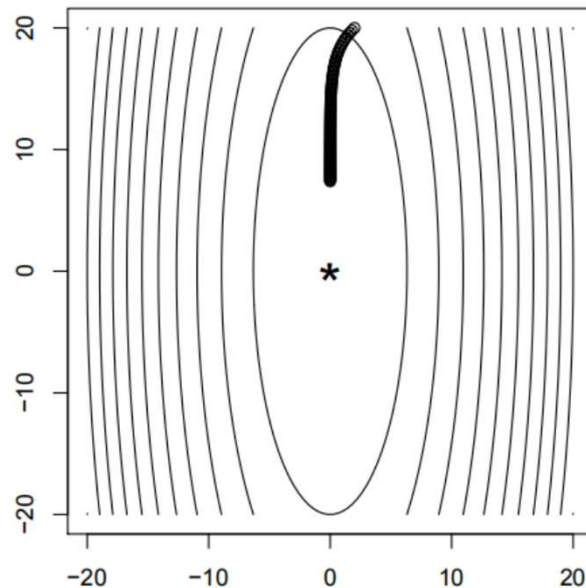
- 不妨令 $f(x) = (10x_1^2 + x_2^2)/2$. 并令每一步 $t_k = t$ for all $k = 1, 2, 3, \dots$.
- 如果步长 t 太大会导致**发散**



gradient descent after 8 steps

• 如何选择步长?

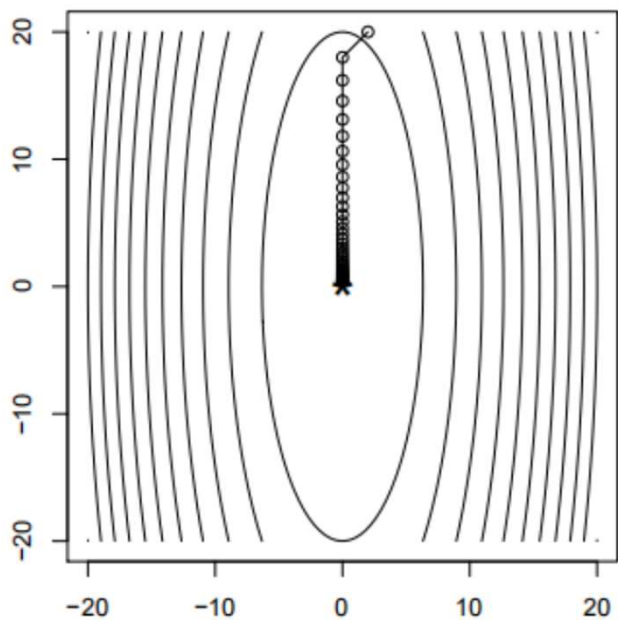
- 不妨令 $f(x) = (10x_1^2 + x_2^2)/2$. 并令每一步 $t_k = t$ for all $k = 1, 2, 3, \dots$.
- 如果步长太小会导致收敛慢



gradient descent after 100 steps

• 如何选择步长?

- 不妨令 $f(x) = (10x_1^2 + x_2^2)/2$. 并令每一步 $t_k = t$ for all $k = 1, 2, 3, \dots$.
- 如果步长 t 选择的合适, 模型可以很好的收敛



gradient descent after 40 steps

• 线搜索

- 精确线搜索: $t = \operatorname{argmin}_{t \geq 0} f(x + t\Delta x)$

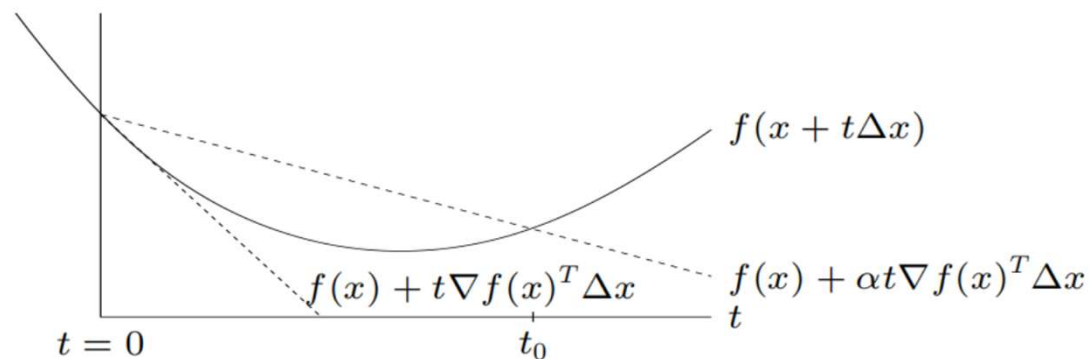
- 回溯线搜索:

- 首先固定参数 $0 < \beta < 1, 0 < \alpha \leq \frac{1}{2}$

- 初始化 $t = 1$, 重复 $t = \beta t$ 直到:

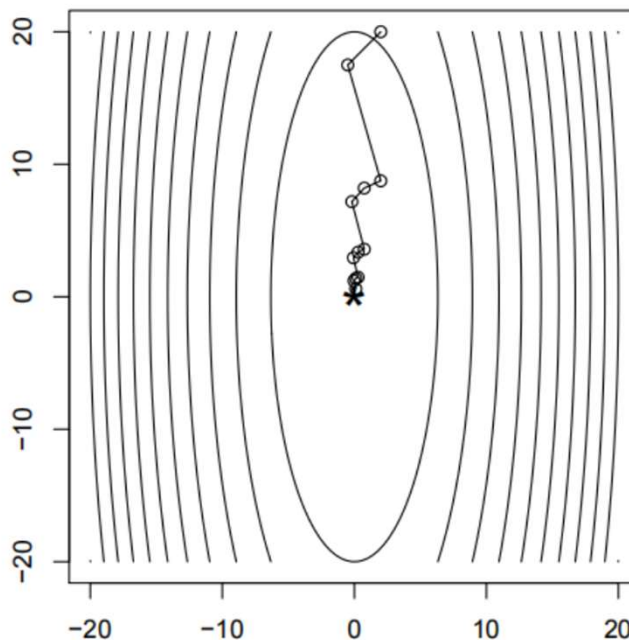
$$f(x + t\Delta x) < f(x) + \alpha t \nabla f(x)^T \Delta x$$

- 几何解释: 直到 $t < t_0$ 回溯



• 线搜索

- 同样的例子，设置 $\alpha = \beta = 0.5$ ，回溯线搜索选择了大致正确的步长
(外部迭代 12 次，总共 40 步)



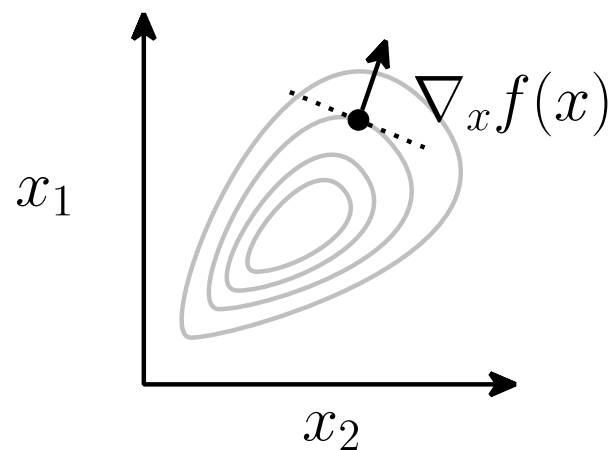
$$\Delta x = -\nabla f(x)$$

• 梯度

- 解决优化问题的关键是函数的梯度表示（多变量情况下的导数）。
- 对于 $f: \mathbb{R}^n \rightarrow \mathbb{R}$, 梯度定义为偏导数的向量

$$\nabla_x f(x) \in \mathbb{R}^n = \begin{bmatrix} \frac{\partial f(x)}{\partial x_1} \\ \frac{\partial f(x)}{\partial x_2} \\ \vdots \\ \frac{\partial f(x)}{\partial x_n} \end{bmatrix}$$

方向为函数 f 增加的“最陡方向”



- 梯度下降法

- 一般的下降法 $\Delta x = -\nabla f(x)$

given a starting point $x \in \text{dom } f$.

repeat

1. $\Delta x := -\nabla f(x)$.
2. *Line search*. Choose step size t via exact or backtracking line search.
3. *Update*. $x := x + t\Delta x$.

until stopping criterion is satisfied.

- 停止准则通常采用 $\|\nabla f(x)\|^2 \leq \epsilon$ 形式
 - 原始的梯度下降简单，但通常非常慢

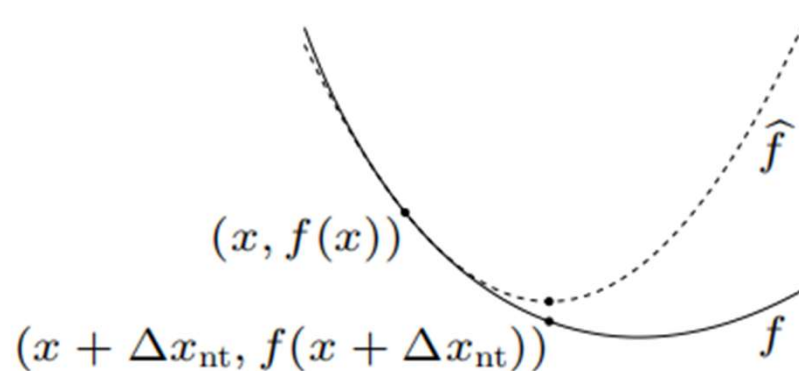
- 牛顿法

$$\Delta x_{nt} = -\nabla^2 f(x)^{-1} \nabla f(x)$$

- 解释:

- $x + \Delta x_{nt}$ 最小化二阶近似

$$\hat{f}(x + v) = f(x) + \nabla f(x)^T v + \frac{1}{2} v^T \nabla^2 f(x) v$$



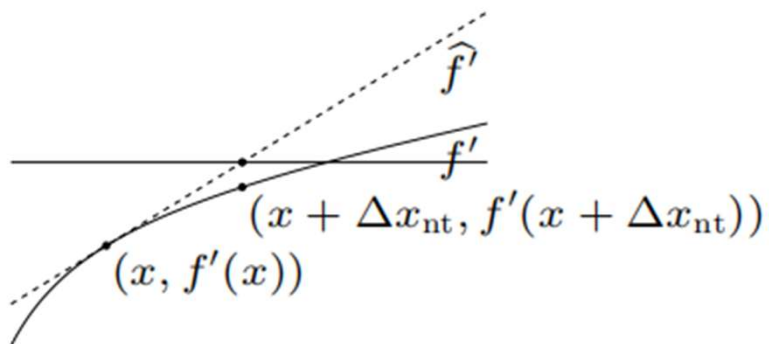
- 牛顿法

$$\Delta x_{nt} = -\nabla^2 f(x)^{-1} \nabla f(x)$$

- 解释:

- $x + \Delta x_{nt}$ 求解线性化的最优性条件

$$\nabla f(x + v) \approx \nabla \hat{f}(x + v) = \nabla f(x) + \nabla^2 f(x)v = 0$$



• 牛顿法

given a starting point $x \in \text{dom } f$, tolerance $\epsilon > 0$.

repeat

1. *Compute the Newton step and decrement.*

$$\Delta x_{\text{nt}} := -\nabla^2 f(x)^{-1} \nabla f(x); \quad \lambda^2 := \nabla f(x)^T \nabla^2 f(x)^{-1} \nabla f(x).$$

2. *Stopping criterion.* **quit** if $\lambda^2/2 \leq \epsilon$.

3. *Line search.* Choose step size t by backtracking line search.

4. *Update.* $x := x + t\Delta x_{\text{nt}}$.

• 牛顿法

given a starting point $x \in \text{dom } f$, tolerance $\epsilon > 0$.

repeat

1. *Compute the Newton step and decrement.*

$$\Delta x_{\text{nt}} := -\nabla^2 f(x)^{-1} \nabla f(x); \quad \lambda^2 := \nabla f(x)^T \nabla^2 f(x)^{-1} \nabla f(x).$$

2. *Stopping criterion.* **quit** if $\lambda^2/2 \leq \epsilon$.

3. *Line search.* Choose step size t by backtracking line search.

4. *Update.* $x := x + t\Delta x_{\text{nt}}$.

• 牛顿衰减

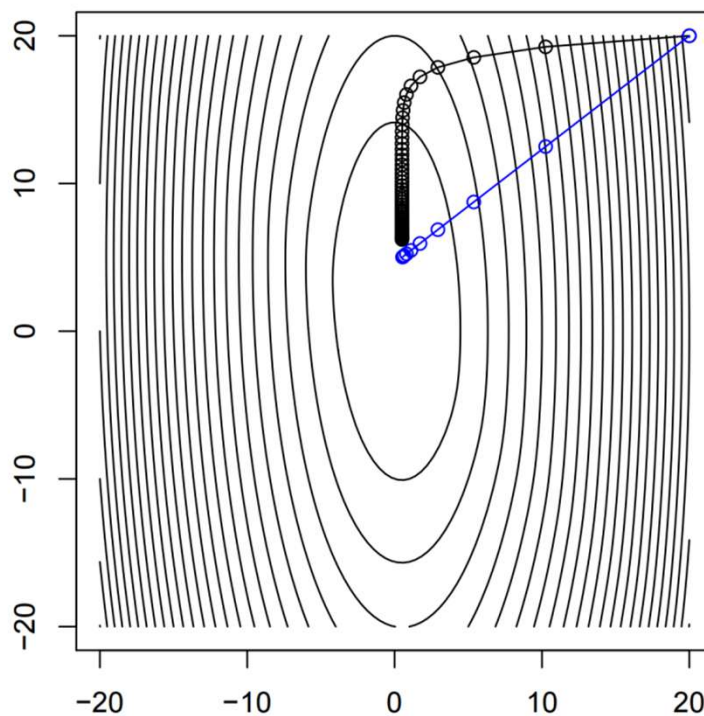
- 在 x 点, 我们将牛顿衰减定义为

$$\lambda(x) = \left(\nabla f(x)^T (\nabla^2 f(x))^{-1} \nabla f(x) \right)^{1/2}$$

- 这是一种求解 x 的最优解近似值的方法

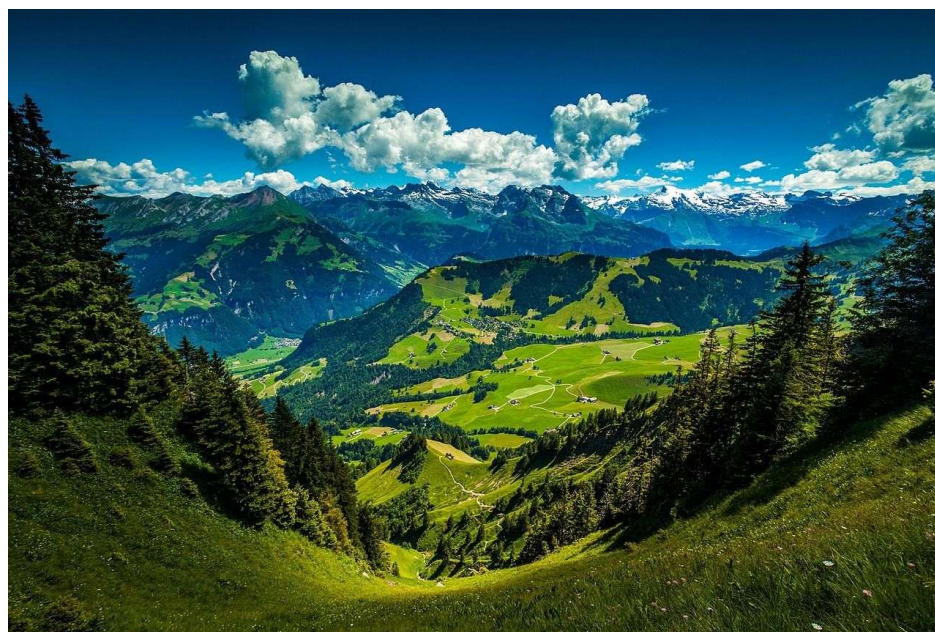
• 牛顿法

- 考虑最小化 $f(x) = \frac{10x_1^2 + x_2^2}{2} + 2 \log(1 + e^{-x_1 - x_2})$
- 对比梯度下降（黑色）和牛顿法（蓝色）（其中，两种方法的每步步长长度大致相同）



- 无约束优化
- 高级梯度优化算法

- 深度学习中的优化器
 - 深度学习中的优化器是一种调整模型参数（如权重和学习率）的算法，以最小化损失函数。它们会影响到训练模型的准确性和速度。
- 几乎所有在深度学习出现的优化问题都是非凸的。

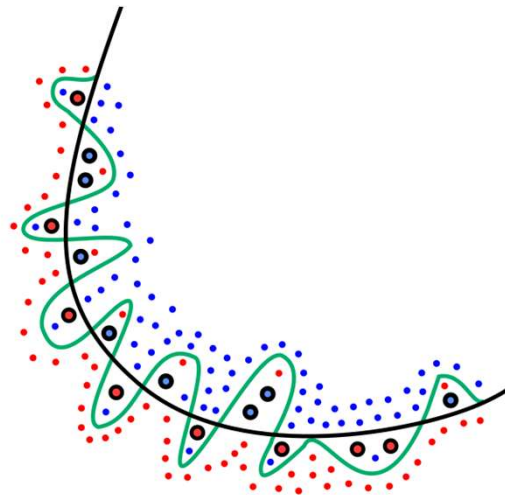


- 优化目标

- 首先定义一个损失函数。
- 使用一种优化算法来尝试最小化损失。在优化过程中，损失函数通常被称为优化问题的目标函数。

- 优化和深度学习的目标是完全不同的

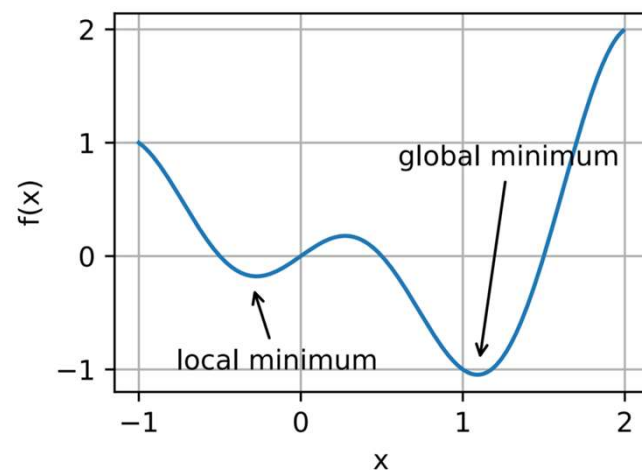
- 给定有限的数据量的前提下，前者主要是最小化一个目标，而后者是寻找一个合适的模型；
- 训练误差和泛化误差一般不同；
- 注意过拟合。



- 深度学习中的优化挑战

- 局部最优解

给定函数 $f(x) = x \cdot \cos(\pi x)$ $x \in [-1.0, 2.0]$

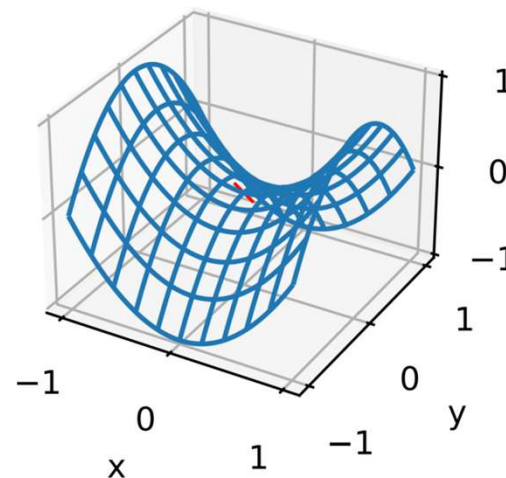
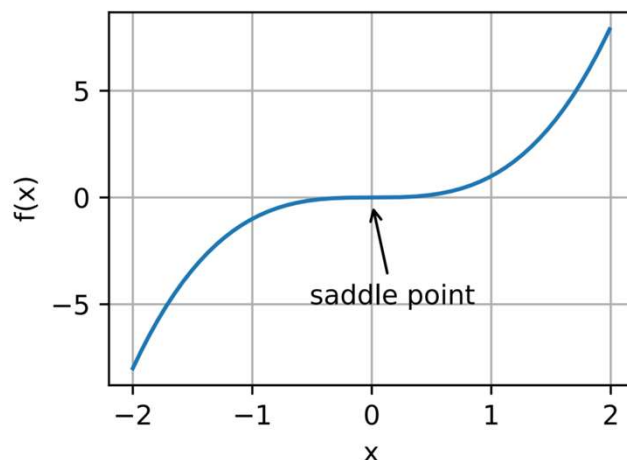


- 深度学习模型的目标函数通常有许多局部最优值。
- 当一个优化问题的数值解接近局部最优时，目标函数的梯度趋于或变为零。

• 深度学习中的优化挑战

• 鞍点

- 鞍点处会出现梯度消失（梯度为0），但它既不是全局也不是局部最小值。
- 考虑函数 $f(x) = x^3$ ， $x = 0$ 时它的一阶导数和二阶导数都为0



- 在更高的维度上的鞍点甚至更加难分辨。考虑一下这个 $f(x,y) = x^2 + y^2$ 函数。它在(0,0)处有鞍点，这是关于 y 的最大值和关于 x 的最小值。

• 深度学习中的优化挑战

• 鞍点

- 我们假设一个函数的输入是一个 k 维向量，其输出是一个标量。
- 该函数的解可以是一个局部最小值，一个局部最大值，或在函数梯度为零的位置上的一个鞍点：

当函数的海森矩阵在零梯度位置的特征值都为正时，该函数有一个局部最小值。

当函数的海森矩阵在零梯度位置的特征值都为负时，该函数有一个局部极大值。

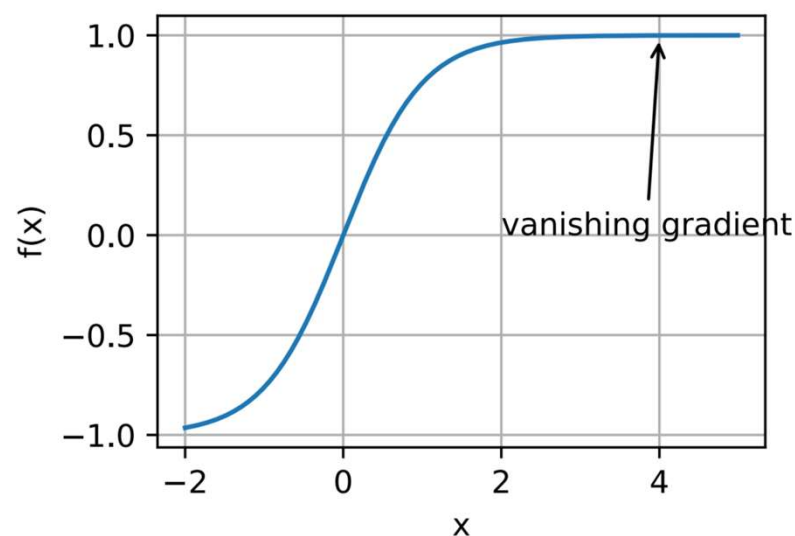
当函数的海森矩阵在零梯度位置的特征值为负和正时，该函数有一个鞍点。

- 简而言之，凸函数是海森的特征值从不为负的函数。但遗憾的是，大多数深度学习问题的损失函数并不属于这一类。

• 深度学习中的优化挑战

• 梯度消失

- 假设我们想最小化 $f(x) = \tanh(x)$ 这个函数，并且我们碰巧就在 $x = 4$ 开始梯度下降。正如我们所看到的，梯度接近于零。更具体地，函数的导数为 $f'(x) = 1 - \tanh^2(x)$ ，也因此如此 $f'(4) = 0.0013$ 。因此，在我们优化取得进展之前将会停滞很长一段时间。



• 梯度下降算法

- 如前所述，梯度下降算法可以降低目标函数的值。

$$x^+ = x - \eta \nabla f(x)$$

- 但是获得“刚好”的学习速度是很棘手的。
 - 如果选择的太小，优化的就很慢。
 - 如果选择的太大，优化过程中就会振荡，甚至可能会发散。
- 如果我们自动决定学习率，或者完全不用选择一个学习率呢？
 - 二阶方法（**牛顿法**）

$$x^+ = x - (\nabla^2 f(x))^{-1} \nabla f(x)$$

- 使用**线搜索**的梯度下降
- 由于**计算成本**，这些方法不能直接应用于深度学习。

- 随机梯度下降

- 考虑最小化函数的平均值

$$\min_x \frac{1}{m} \sum_{i=1}^m f_i(x)$$

- 由于 $\nabla \sum_{i=1}^m f_i(x) = \sum_{i=1}^m \nabla f_i(x)$, 这相当于梯度下降会重复以下过程:

$$x^{(k+1)} = x^{(k)} - \eta_{k+1} \cdot \frac{1}{m} \sum_{i=1}^m \nabla f_i(x^{(k)}), k = 0, 1, 2, \dots$$

- 相比之下, 随机梯度下降或SGD (或增量梯度下降) :

$$x^{(k+1)} = x^{(k)} - \eta_{k+1} \cdot \nabla f_{i_{k+1}}(x^{(k)}), k = 0, 1, 2, \dots$$

其中 $i_k \in \{1, \dots, m\}$ 是从迭代次数 k 中选择的部分索引

• 随机梯度下降

• 在迭代次数 k 中选择索引 i_k 的两条法则:

- **随机选取**: 随机均匀地选取 $i_k \in \{1, \dots, m\}$
- **循环选取**: $i_k = 1, 2, \dots, m, 1, 2, \dots, m, \dots$

• 随机选取在实践中更为常见。对于随机选取有:

$$\mathbb{E}[\nabla f_{i_k}(x)] = \nabla f(x)$$

• 所以我们可以把SGD看作是使用每一步梯度的无偏估计。

• **SGD的主要优点:**

- 迭代成本与 m （批次的数量）无关。
- 也可以在内存使用量方面大大节省开支。

- 小批量梯度下降

- 同样常见的是小批量随机梯度下降，其中我们选择一个随机子集 $I_k \subseteq \{1, 2, \dots, m\}$, $|I_k| = b \ll m$, 重复:

$$x^{(k+1)} = x^{(k)} - \eta_{k+1} \cdot \frac{1}{b} \sum_{i \in I_k} \nabla f_i(x^{(k)}), k = 0, 1, 2, \dots$$

- 同样，用一个无偏的估计来近似全梯度：

$$\mathbb{E} \left[\frac{1}{b} \sum_{i \in I_k} \nabla f_i(x) \right] = \nabla f(x)$$

- 使用小批量数据可以将方差减小到原来的 $1/b$ ，但计算成本也增加了 b 倍。
- 可以通过一次应用到一个小批量数据中来提高计算效率。

- 例子：随机逻辑回归

- 给定 $(x_i, y_i) \in \mathbb{R}^p \times \{0, 1\}$, $i = 1, \dots, n$, 逻辑回归损失为:

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n -y_i \log p_i - (1 - y_i) \log(1 - p_i)$$

- 逻辑回归模型为:

$$p_i = \frac{1}{1 + \exp(-x_i^T \beta)}$$

$$\begin{aligned} \mathcal{L} &= \frac{1}{n} \sum_{i=1}^n -y_i \log \frac{\exp(x_i^T \beta)}{1 + \exp(x_i^T \beta)} - (1 - y_i) \log \left(\frac{1}{1 + \exp(x_i^T \beta)} \right) \\ &= \frac{1}{n} \sum_{i=1}^n -y_i x_i^T \beta + \log(1 + \exp(x_i^T \beta)) \end{aligned}$$

- 优化问题为:

$$\min_{\beta} \frac{1}{n} \sum_{i=1}^n (-y_i x_i^T \beta + \log(1 + \exp(x_i^T \beta)))$$

- 例子：随机逻辑回归

$$\min_{\beta} \frac{1}{n} \sum_{i=1}^n (-y_i x_i^T \beta + \log(1 + \exp(x_i^T \beta)))$$

- 加上正则化：

$$\min_{\beta} \frac{1}{n} \sum_{i=1}^n (-y_i x_i^T \beta + \log(1 + \exp(x_i^T \beta))) + \frac{\lambda}{2} \|\beta\|_2^2$$

- 为了方便表示，令：

$$f(\beta) = \frac{1}{n} \sum_{i=1}^n f_i(\beta), \quad f_i(\beta) = -y_i x_i^T \beta + \log(1 + \exp(x_i^T \beta)) + \frac{\lambda}{2} \|\beta\|_2^2$$

- 例子：随机逻辑回归

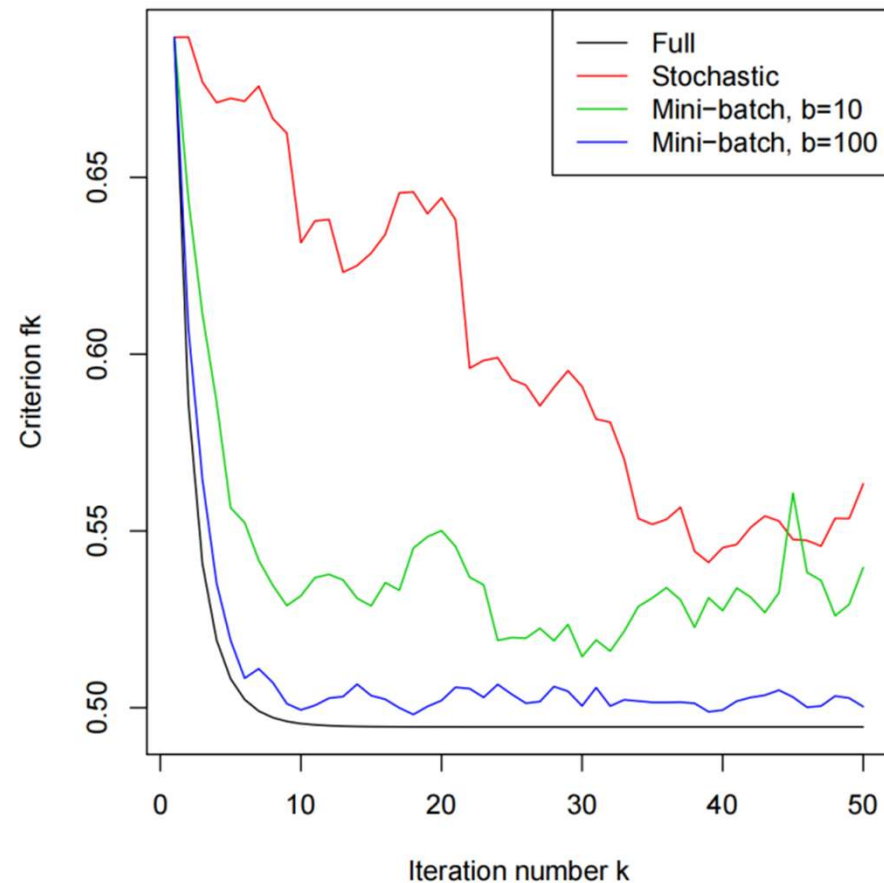
- 梯度计算 $\nabla f(\beta) = \frac{1}{n} \sum_{i=1}^n (y_i - p_i(\beta)) x_i + \lambda \beta$ 在 n 大小适中时是可行的，但在 n 较大时则不可行

- 方法比较

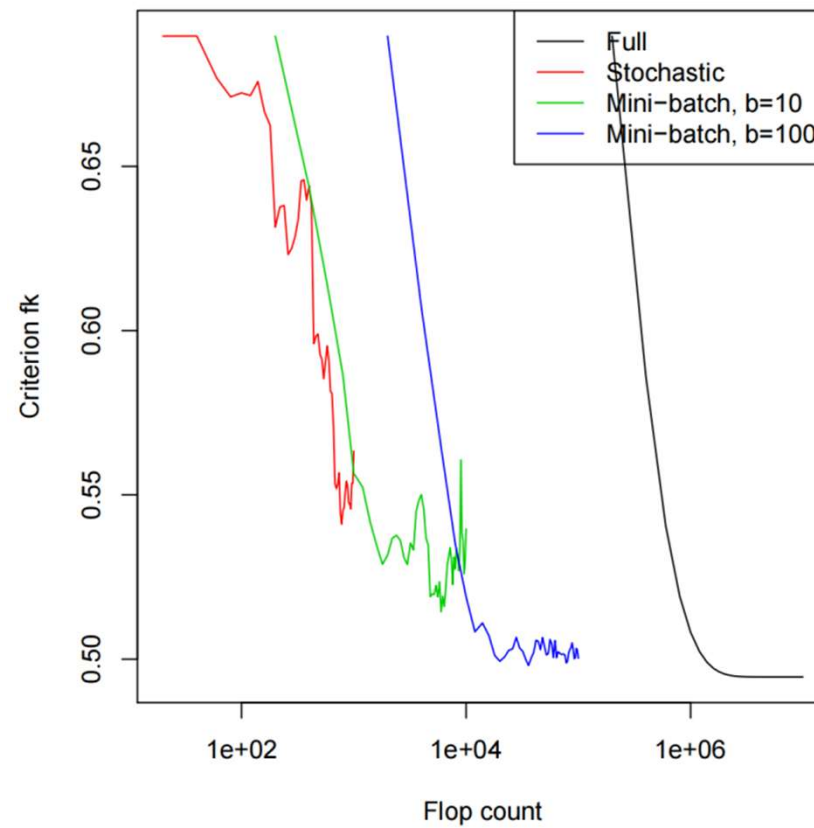
- 一次批更新的成本为 $O(np)$
- 一次小批量更新的成本为 $O(bp)$
- 一次随机更新的成本为 $O(p)$

- 例子：随机逻辑回归

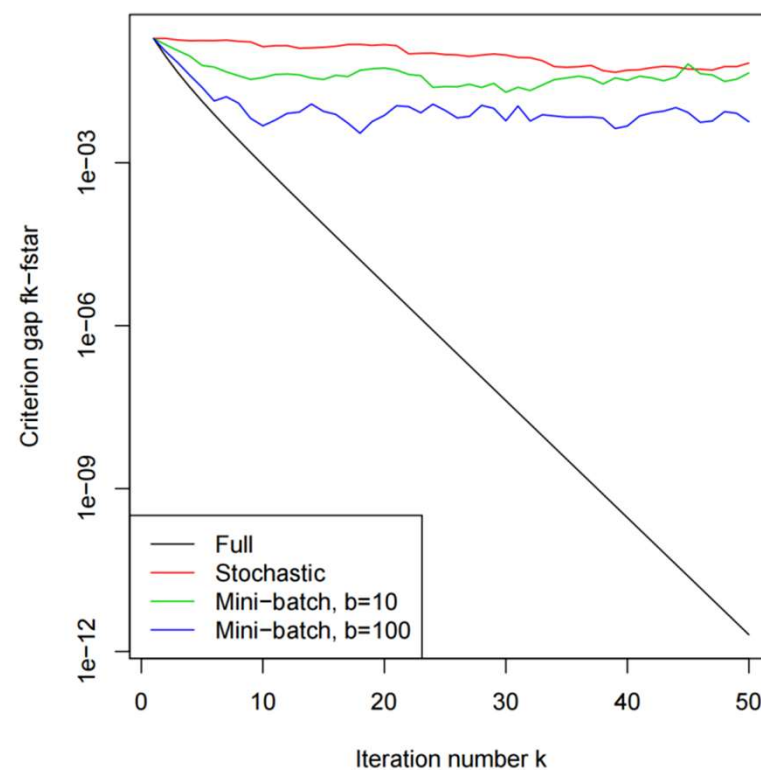
- 例如 $n = 10,000$, $p = 20$, 所有方法都使用固定的步长：



- 例子：随机逻辑回归
 - 按照浮点运算次数来进行对比

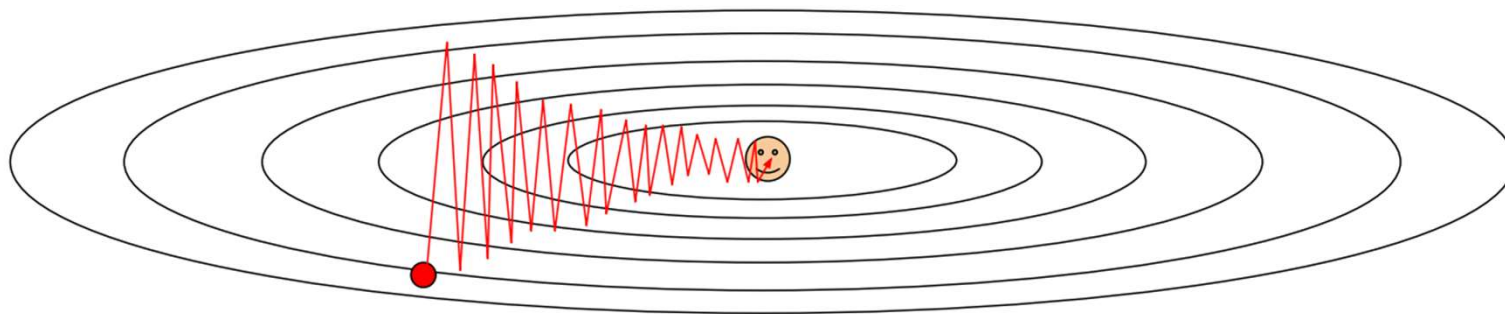


- 例子：随机逻辑回归
 - 最后，看看次优性差距（在对数尺度上）：



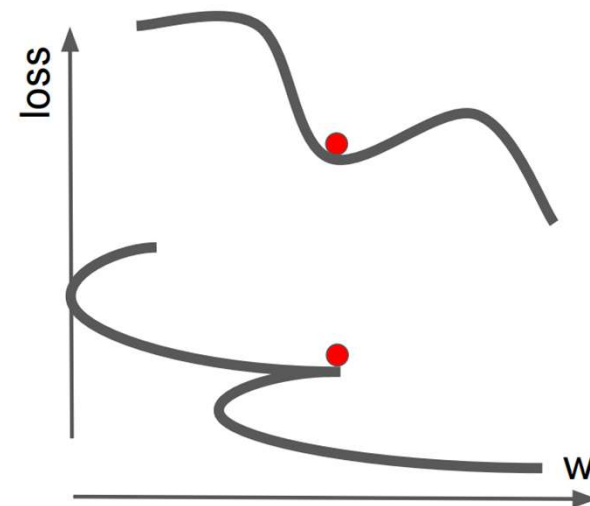
• SGD的问题

- 如果损失在一个方向上快速变化，在另一个方向上变化缓慢，梯度下降会怎么做？
 - 在梯度较小的维度上进展缓慢，在梯度较大的方向上会出现抖动。



• SGD的问题

- 如果损失函数有局部最小值或鞍点呢？
 - 梯度为零，梯度下降被卡住
- 鞍点在高维度中更为常见
- 梯度来自于小批次，所以它们可能会很嘈杂



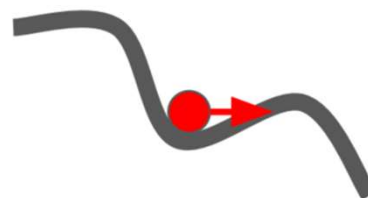
- 动量

- 动量方法可以解决我们之前提到的SGD出现的问题

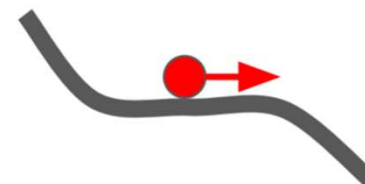
$$v^{k+1} = \beta v^k + \nabla f(x^k)$$

$$x^{k+1} = x^k - \eta^{k+1} v^{k+1}$$

Local Minima

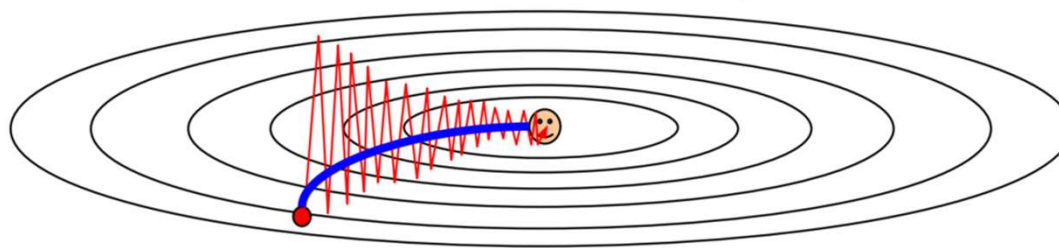


Saddle points



- 动量有助于加速SGD在相关方向上的速度并减少震荡。

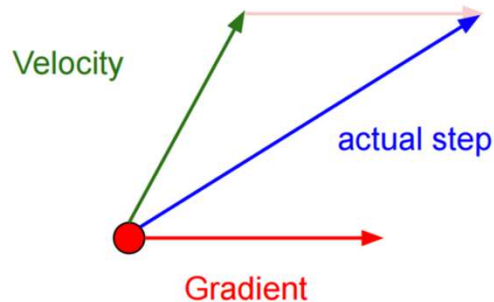
Poor Conditioning



• Nesterov动量法

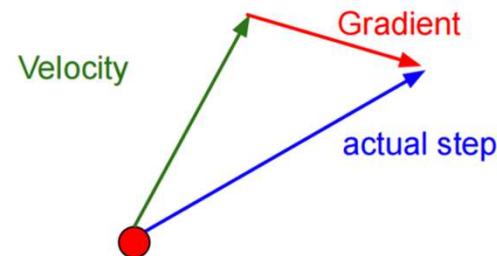
- 根据预测位置的更新可以防止我们走得太快，并提高响应性。
- Momentum update VS Nesterov Momentum

Momentum update:



结合当前点的gradient与velocity
得到用于更新权重的方法

Nesterov Momentum



先预测使用当前速度更新会到达的位置，然
后在该位置计算梯度，并将此gradient与
velocity结合起来以确定实际的更新方向

• Nesterov动量法

- 通过计算我们参数的预估未来位置梯度（而不是当前参数的梯度），能够有效地进行前瞻

$$\begin{aligned}v^{k+1} &= \beta v^k + \eta^{k+1} \nabla f(x^k - \beta v^k) \\x^{k+1} &= x^k - v^{k+1}\end{aligned}$$

- 另一种写法 $\tilde{x}^k = x^k - \beta v^k$

$$\begin{aligned}v^{k+1} &= \beta v^k + \eta^{k+1} \nabla f(\tilde{x}^k) \\ \tilde{x}^{k+1} &= \tilde{x}^k - v^{k+1} + \beta(v^k - v^{k+1})\end{aligned}$$

• Adagrad

- 在学习率逐渐减小的情况下，可能会遇到这样的情形：常见特征的参数相对快速地收敛到它们的最优值；而对于不常见的特征，它们还没有足够多地进行优化，因此其还未收敛到最优值。

- Adagrad算法会根据参数自适应地调整学习率

$$g^k = \nabla f(x^k)$$

$$s^{k+1} = s^k + (g^k)^2$$

$$x^{k+1} = x^k - \frac{\eta}{\sqrt{s^{k+1}} + \epsilon} \odot g^k$$

- 通常把 s^0 初始化为0
- 注意 s^k 是和 x^k 同维度的向量，不是标量

• Adagrad

- 这两个好处：
 - 不在需要决定梯度何时足够大
 - 自动根据梯度的大小进行调整。
- Adagrad非常适合处理稀疏数据。

问题：经过多轮迭代，Adagrad的步长会发生什么变化？

答：减小为零。该算法不再能够获得新的知识。

注意是向量，不是标量！

1. The update rule

$$w_i^{(t+1)} = w_i^{(t)} - \frac{\eta}{\sqrt{\sum_{\tau=1}^t g_{\tau,i}^2}} g_{t,i},$$

is the composite mirror descent variety of ADAGRAD. Take a look at equation 4 (or 23) from the paper. When there's no regularization term, the update can be derived (take derivative and set to 0) as:

$$w_{t+1} = w_t - \eta \text{diag}(G)^{-1/2} g_t.$$

Notice that $\text{diag}(G)_{ii}^{-1/2} = \frac{1}{\sqrt{\sum_{\tau=1}^t g_{\tau,i}^2}}$ and that multiplying a diagonal matrix by a vector amounts to element-wise multiplication.

2. η is a constant step size which determines the size of the step at each update. I would try several choices and compare them.
3. Changing an algorithm from SGD to ADAGRAD just requires plugging in your gradient values. That is, you need to keep track of the sum of squared gradient elements for the term in the denominator.

- RMSProp: 被称为 “Leaky AdaGrad”
 - 常数 $\epsilon > 0$ 通常被设置为 1×10^{-6} , 以确保不会因为除零或过大的步长而遇到问题。

$$s^{k+1} = \lambda s^k + (1 - \lambda)(g^k)^2$$

$$x^{k+1} = x^k - \frac{\eta}{\sqrt{s^{k+1} + \epsilon}} \odot g^k$$

• Adam

- 自适应矩估计 (Adam) 算法结合了以上所有技术, 形成了一个高效的学习算法。

- 第一矩 (均值) :

$$v^{k+1} = \beta_1 v^k + (1 - \beta_1) g^k$$

- 第二矩 (非中心化的方差) :

- 这里 β_1 和 β_2 是非负的权重参数。它们的常见选择是 $\beta_1 = 0.9$ 和 $\beta_2 = 0.999$ 。

$$s^{k+1} = \beta_2 s^k + (1 - \beta_2) (g^k)^2$$

• Adam

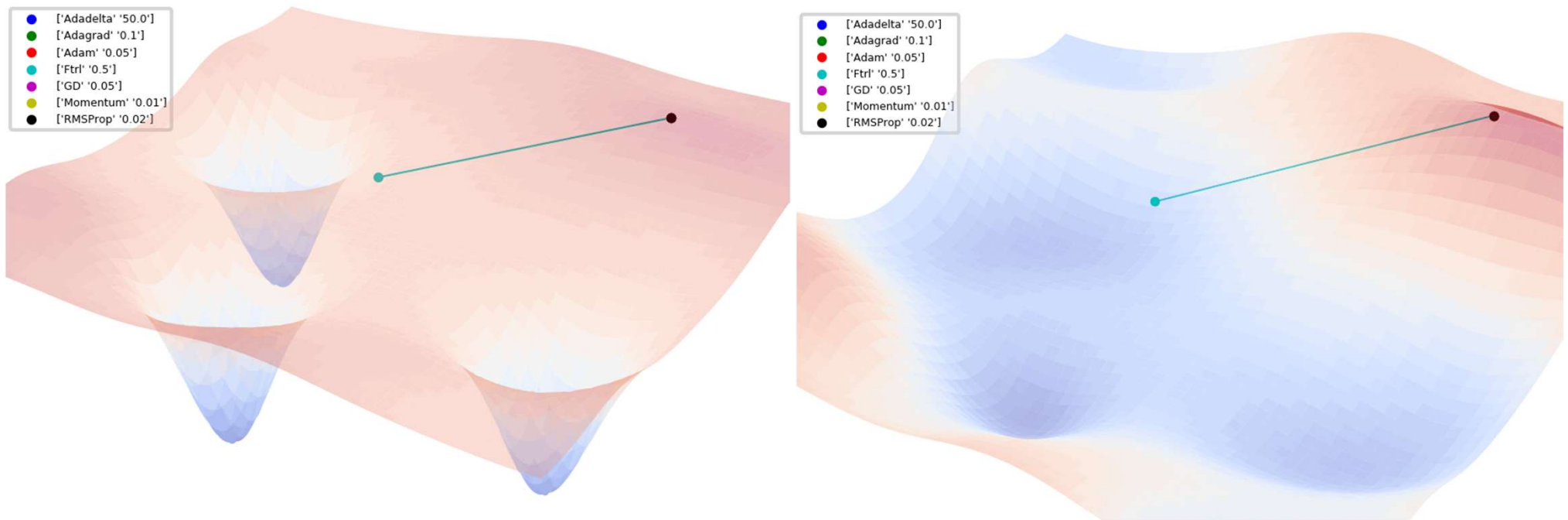
- 值得注意的是，如果我们初始化 $v^0 = 0, s^0 = 0$ ，那么在最初时算法会对较小的梯度值有过分的偏爱。
- 我们可以通过计算偏差校正后的第一矩和第二矩估计来抵消这些偏爱：

$$\hat{s}^k = \frac{s^k}{1 - (\beta_1)^k}$$
$$\hat{v}^k = \frac{v^k}{1 - (\beta_2)^k}$$

- 最后，我们可以得到变量更新的式子：

$$x^{k+1} = x^k - \frac{\eta}{\sqrt{\hat{s}^{k+1} + \epsilon}} \hat{v}^k$$

• 算法可视化

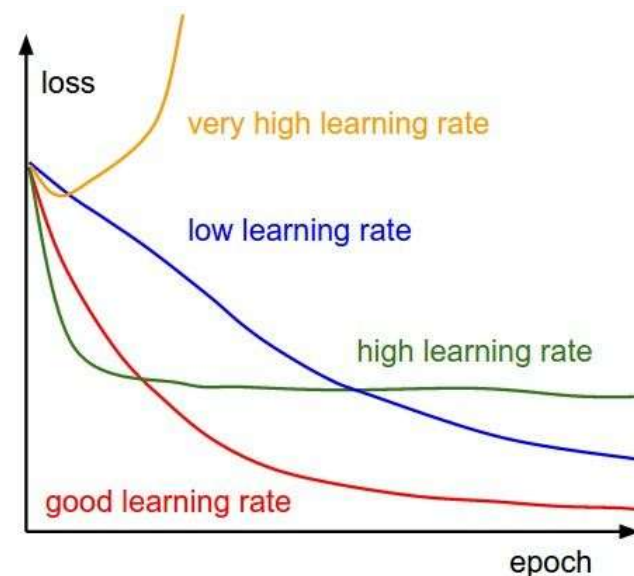


<https://github.com/Jaewan-Yun/optimizer-visualization>

- 要阅读更多内容，请访问：
 - <https://www.ruder.io/optimizing-gradient-descent/#visualizationofalgorithms>
 - https://d2l.ai/chapter_optimization/
 - <https://distill.pub/2017/momentum/>

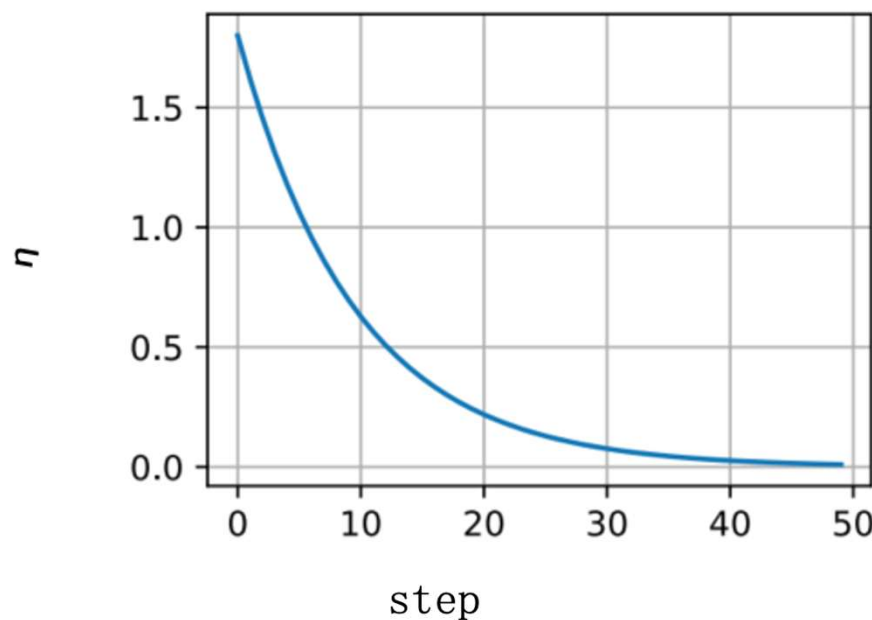
• 学习率设置

- 如果学习率太大，优化就会发散，如果太小，它就需要太长的时间来训练，否则最终会得到一个次优的结果
- 如果学习率较大，最初可以快速优化，但最终可能就会在最小值附近震荡，因此无法达到最优状态
- 开始时的大学习率可能没有好处，特别是由于初始参数集是随机的。最初的更新方向相当可能也毫无意义。



• Factor Scheduler

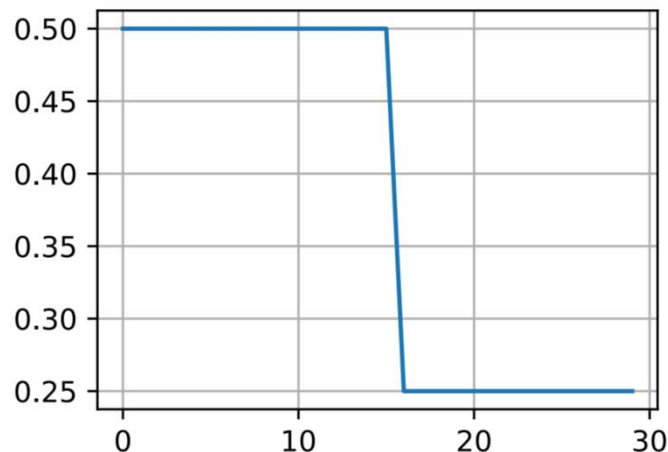
- 一个常用的衰减方法是乘法衰减，即 $\eta^{k+1} = \eta^k \cdot \alpha$, for $\alpha \in (0,1)$ 。为了防止学习率衰减到一个合理的下限以下，更新方程常常被修改为 $\eta^{k+1} = \max(\eta_{min}, \eta^k \cdot \alpha)$ 。



$$\alpha = 0.9$$
$$\eta^0 = 2.0$$

• Multi Factor Scheduler

- 在深度网络训练中，一个常见的策略是分段学习率，每隔一段时间就减少一定的比例。通常，给定一个时间集合来决定何时降低学习率，比如 $s = \{5, 10, 20\}$ ， $\eta^{k+1} = \eta^k \cdot \alpha$ whenever $k \in s$ 。如果假设在每个步骤中学习率减半，我们可以按照以下方式实施这个策略。

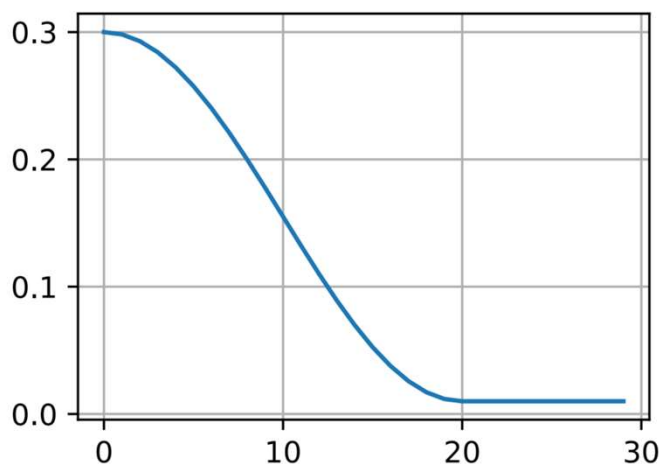


```
scheduler = lr_scheduler.MultiFactorScheduler(step=[15, 30], factor=0.5,  
                                              base_lr=0.5)  
d2l.plot(np.arange(num_epochs), [scheduler(t) for t in range(num_epochs)])
```

• Cosine Scheduler

$$\eta^{t+1} = \eta^T + \frac{\eta^0 - \eta^T}{2} \left(1 + \cos\left(\frac{\pi t}{T}\right)\right)$$

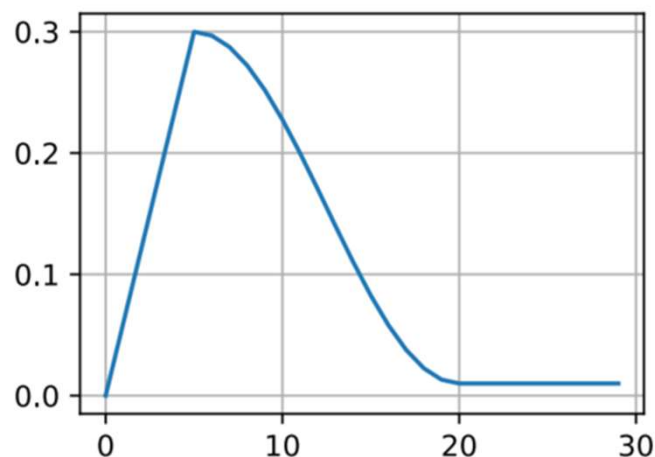
- 这里 η^0 是初始学习率， η^T 是当 T 时的目标学习率。此外，对于 $t > T$ ，我们只需将值固定到 η^T 而不再增加它。在下面的示例中，我们设置了最大更新步数 $T = 20$ 。



```
scheduler = lr_scheduler.CosineScheduler(max_update=20, base_lr=0.3,  
                                         final_lr=0.01)  
d2l.plot(np.arange(num_epochs), [scheduler(t) for t in range(num_epochs)])
```

• Warmup

- 在某些情况下，初始化参数不足以得到良好的解。这对某些高级网络设计来说尤其棘手，可能导致不稳定的优化结果。对此，一方面，我们可以选择一个足够小的学习率，从而防止一开始发散，然而这样进展太缓慢。另一方面，较高的学习率最初就会导致发散。
- 解决这种困境的一个相当简单的解决方法是使用warmup，在此期间学习率将增加至初始最大值，然后冷却直到优化过程结束。为了简单起见，通常使用线性递增。



```
scheduler = lr_scheduler.CosineScheduler(20, warmup_steps=5, base_lr=0.3,  
                                         final_lr=0.01)  
d2l.plot(np.arange(num_epochs), [scheduler(t) for t in range(num_epochs)])
```

- Stephen Boyd, EE364a: Convex Optimization I, Stanford University
- Aditi Raghunathan, Tuomas Sandholm, 15-780 :Graduate Artificial Intelligence, Carnegie Mellon University, Spring 2023
- Ryan Tibshirani, "Convex Optimization" , Carnegie Mellon University, Fall 2019
- Feifei Li, Yunzhu Li, Ruochan Gao, CS231n: Deep Learning for Computer Vision, Stanford University, Spring 2023
- Zhang A, Lipton Z C, Li M, et al. Dive into deep learning[J]. arXiv preprint arXiv:2106.11342, 2021.

THANK YOU!

Jifeng Dai
Department of Electronic Engineering
daijifeng@tsinghua.edu.cn

